Dynamic Rental Management System

Problem Statement:-

Managing rental variations manually or with a rigid system leads to certain errors, inefficiency, and difficulty in scaling. This system will dynamically calculate rental costs, manage availability, enforce return policies, and handle exceptional scenarios such as late returns or invalid bookings.

Target Users:-

Small rental businesses, Vehicle rental services, Equipment rental providers, Property rental managers.

SYSTEM DESIGN

User → Main → rentalService → model → CSV File

Key Classes

1. **rentableItems (base)** - acts as base class
2. **rentTransaction** - represents rental records, stores item, duration, cost
3. **rentalService** - manages items, customer, transaction and handles rent and return operations.
4. **customer** - stores customer details and maintains rental history

Package Structure

```
└── com
    └── parijat
        └── rental
            ├── exceptions
            ├── main
            ├── model
            └── service
```

OOP Concepts Used

- ***Abstraction:*** rentableItems class
- ***Inheritance:*** FourWheeler, TwoWheeler, Room, Accessories
- ***Polymorphism:*** calculateRentalCost()
- ***Encapsulation:*** Private fields like rentalHistory

TECHNICAL HIGHLIGHT

Core Logic:-
rentItem()

- It is the core logic of the program as it serves:

1. Main purpose of the system : renting
2. Uses Collection (Hash Map)
3. Polymorphism with TotalCost()
4. Exception Handling

```java
public void rentItem(String transactionId, String customerId, String itemId, int days)
        throws CustomerNotFoundException, ItemNotFoundException,
               ItemNotAvailableException, InvalidRentDurationException{
    if (days<=0) {
        throw new InvalidRentDurationException(message: "Days must be greater than zero");
    }
    if (!customers.containsKey(customerId)) {
        throw new CustomerNotFoundException(message: "Customer not found");
    }
    if (!items.containsKey(itemId)) {
        throw new ItemNotFoundException(message: "Item not found");
    }
    rentableItems item =items.get(itemId);
    if (!item.isAvailable()) {
        throw new ItemNotAvailableException(message: "Item is not available");
    }
    rentTransaction transaction =new rentTransaction(transactionId, item, days);
    System.out.println("Total Rental Cost: " + transaction.TotalCost());
    item.markAsRented();
    activeRentals.put(transactionId, transaction);
    customer customer =customers.get(customerId);
    customer.addTransaction(transaction);
    saveTransactionToFile(transaction);
    item.markAsRented();
    activeRentals.put(transactionId, transaction);
    customer.addTransaction(transaction);
}
```

Output Flow with Live Demo

1. MENU:

```
--------------- Rental Management System ---------------
1. Add Item
2. Register Customer
3. Rent Item
4. Return Item
5. Display Items
6. Exit
```

2.ADD

```
Enter choice: 1
Enter item type (1.Four Wheeler, 2.Two Wheeler, 3.Room, 4.Accessories): 1
Enter Item ID: Alto_800-01
Enter Base Rate: 4000
Item Added Successfully!!
```

3.REGISTER CUSTOMER:

```
Enter choice: 2
Enter Customer ID: 101
Enter Customer Name: Parijat
Customer registered!
```

4.RENT ITEM:

```
Enter choice: 3
Enter Transaction ID: 001
Enter Customer ID: 102
Enter Item ID: Alto_800-01
Enter Days: 3
Total Rental Cost: 12150.0
Item rented successfully!
```

5.DISPLAY ITEMS:

```
Enter choice: 5
Item ID: Alto_800-01
Available ? : No
```

6.RETURN ITEMS:

```
Enter choice: 4
Enter Transaction ID: 001
Item returned successfully!
```